Loading Libraries

```
import os
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers
import keras
from keras.preprocessing import sequence
from keras.callbacks import
ModelCheckpoint,ReduceLROnPlateau,EarlyStopping
```

Loading dataset

```
path_to_file = tf.keras.utils.get_file('shakespeare.txt',
'https://storage.googleapis.com/download.tensorflow.org/data/shakespea
re.txt')
```

```
Downloading data from
https://storage.googleapis.com/download.tensorflow.org/data/shakespear
e.txt
1122304/1115394 [==============================] - 0s 0us/step
1130496/1115394 [==============================] - 0s 0us/step
```

```
text = open(path_to_file, 'rb').read().decode(encoding='utf-8')
# length of text is the number of characters in it
print ('Length of text: {} characters'.format(len(text)))
```

```
Length of text: 1115394 characters
```

Creating vocabulary and converting char to text and text to char

```
vocab = sorted(set(text))
char2idx = {u:i for i,u in enumerate(vocab)}
idx2char = np.array(vocab)

def text_to_int(text):
  return np.array([char2idx[c] for c in text])

text_as_int = text_to_int(text)

print("Text: ",text[:13])
print("Encoded: ",text_to_int(text[:13]))
```

```
Text:  First Citizen
Encoded:  [18 47 56 57 58  1 15 47 58 47 64 43 52]
```

```
def int_to_text(ints):
  try:
    ints = ints.numpy()
  except:
    pass
  return ''.join(idx2char[ints])
```

```
print(int_to_text(text_as_int[:13]))
```

```
First Citizen
```

Creating training examples

```
seq_length = 100
epochs = len(text)//(seq_length+1)

char_dataset = tf.data.Dataset.from_tensor_slices(text_as_int)
```

```
2022-03-21 12:36:29.125123: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-21 12:36:29.211598: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-21 12:36:29.212320: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-21 12:36:29.215323: I
tensorflow/core/platform/cpu_feature_guard.cc:142] This TensorFlow
binary is optimized with oneAPI Deep Neural Network Library (oneDNN)
to use the following CPU instructions in performance-critical
operations:  AVX2 AVX512F FMA
To enable them in other operations, rebuild TensorFlow with the
appropriate compiler flags.
2022-03-21 12:36:29.216047: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-21 12:36:29.216690: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-21 12:36:29.217327: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-21 12:36:31.245054: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-21 12:36:31.245803: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
```

```
2022-03-21 12:36:31.246481: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-21 12:36:31.247058: I
tensorflow/core/common_runtime/gpu/gpu_device.cc:1510] Created
device /job:localhost/replica:0/task:0/device:GPU:0 with 15403 MB
memory:  -> device: 0, name: Tesla P100-PCIE-16GB, pci bus id:
0000:00:04.0, compute capability: 6.0
```

```
seq = char_dataset.batch(seq_length+1 , drop_remainder = True)
```

```
def split_input_target(chunk):
  input_text = chunk[:-1]
  target_text = chunk[1:]
  return input_text,target_text

dataset = seq.map(split_input_target  )
```

Printing the text

```
for x,y in dataset.take(2):
  print("\n\nExample\n")
  print("Input")
  print(int_to_text(x))
  print("\nOutput")
  print(int_to_text(y))
```

```
Example

Input
First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You

Output
irst Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You
```

```
Example

Input
are all resolved rather to die than to famish?

All:
Resolved. resolved.

First Citizen:
First, you

Output
re all resolved rather to die than to famish?

All:
Resolved. resolved.

First Citizen:
First, you k
```

```
2022-03-21 12:36:31.698199: I
tensorflow/compiler/mlir/mlir_graph_optimization_pass.cc:185] None of
the MLIR Optimization Passes are enabled (registered 2)
```

Batch training

```
BATCH_SIZE = 64
VOCAB_SIZE = len(vocab)
EMBEDDING_DIM = 256
RNN_UNITS = 1024

BUFFER_SIZE = 1000
data = dataset.shuffle(BUFFER_SIZE).batch(BATCH_SIZE,drop_remainder =
True)
```

Model

```
def build_model(vocab_size, embedding_dim,rnn_units,batch_size):
  model = tf.keras.Sequential([

tf.keras.layers.Embedding(vocab_size,embedding_dim,batch_input_shape =
[batch_size,None]),
                              tf.keras.layers.LSTM(rnn_units,
return_sequences = True, stateful = True, recurrent_initializer =
'glorot_uniform'),
                              tf.keras.layers.Dense(vocab_size)

  ])
  return model
```

```
model = build_model(VOCAB_SIZE,EMBEDDING_DIM,RNN_UNITS,BATCH_SIZE)
model.summary()
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding (Embedding)        (64, None, 256)           16640
_________________________________________________________________
lstm (LSTM)                  (64, None, 1024)          5246976
_________________________________________________________________
dense (Dense)                (64, None, 65)            66625
=================================================================
Total params: 5,330,241
Trainable params: 5,330,241
Non-trainable params: 0
_________________________________________________________________
```

Loss function

```
for input_example_batch,target_example_batch in data.take(1):
  example_batch_predictions = model(input_example_batch)
  print(example_batch_predictions.shape)
```

```
2022-03-21 12:36:32.958287: I
tensorflow/stream_executor/cuda/cuda_dnn.cc:369] Loaded cuDNN version
8005

(64, 100, 65)
```

```
print(len(example_batch_predictions))
print(example_batch_predictions)
```

```
64
tf.Tensor(
[[[-6.05337322e-03  6.06625900e-03 -1.69885898e-05 ... -2.28285021e-03
   -4.21570567e-03 -2.66753044e-03]
  [-5.23816422e-03  4.61793691e-03 -1.17608290e-02 ... -1.44323579e-03
   -1.82086905e-03  9.46905231e-04]
  [-9.87576228e-03  9.61039402e-03 -9.28576291e-03 ... -3.22117563e-03
   -4.76195756e-03 -1.42869446e-03]
  ...
  [-7.27287587e-03  3.84986866e-03 -1.83184128e-02 ... -1.59163109e-03
    3.25189671e-03 -7.54578412e-03]
  [-5.92862722e-03  1.10684661e-03 -1.84623897e-02 ...  7.60537223e-04
   -2.37168511e-04 -7.05897668e-03]
  [-4.47580125e-03 -3.21667467e-04 -8.28207284e-03 ... -8.99060629e-04
    5.49715571e-03 -2.65536201e-03]]

 [[-1.52882468e-03 -6.09554874e-04 -1.14973858e-02 ...  2.86858471e-04
    1.18946086e-03  2.47103651e-03]
  [-2.78395624e-03 -3.76937678e-04 -8.24265555e-03 ... -3.45862075e-03
```

```
   1.61510019e-03 -6.04799949e-03]
  [-2.18659267e-03 -5.66895003e-04 -1.05133206e-02 ... -3.46548320e-03
   -1.17268937e-03 -6.72304910e-03]
  ...
  [-8.87280144e-03 -1.45599269e-03 -1.23193711e-02 ... -2.21315306e-04
    4.11003456e-03 -1.10368768e-03]
  [-1.06974598e-02 -3.56880343e-03 -1.18885674e-02 ... -7.71359261e-03
    8.86504911e-03 -4.26188984e-04]
  [-1.17448736e-02 -1.16775353e-02 -1.17143244e-02 ... -9.67695378e-03
    9.78530478e-03 -6.21114112e-03]]

 [[-1.52962026e-03 -5.02534024e-03  1.11567276e-03 ...  1.07207196e-03
    2.58581177e-03 -1.52321730e-03]
  [-3.51003185e-03 -1.13792829e-02  3.26209702e-04 ... -3.09173600e-03
    3.54925822e-03 -6.47638412e-03]
  [-1.74221490e-03 -5.70392609e-03  1.74433994e-03 ... -5.22640161e-03
    9.07659531e-03 -4.65117022e-03]
  ...
  [-8.72506946e-03 -2.52457871e-03 -1.25757363e-02 ... -8.14482197e-03
    8.05898197e-03 -5.13158878e-03]
  [-1.10607464e-02 -1.07516814e-02 -1.07247299e-02 ... -9.61141568e-03
    9.16820578e-03 -1.06650470e-02]
  [-9.18295048e-03 -9.91749391e-03 -1.02009615e-02 ... -6.00366341e-03
    6.59788307e-03 -2.64918595e-03]]

 ...

 [[ 2.38951948e-03 -4.51413076e-03  2.23111967e-03 ... -2.27145036e-03
   -5.65035921e-03 -7.29394960e-05]
  [-6.56130724e-05  3.43350926e-04 -1.78525271e-03 ... -2.32528127e-03
   -3.17010237e-03 -4.00089566e-03]
  [ 3.69286817e-03  3.66525259e-04 -8.47558491e-04 ... -3.99290211e-03
   -3.88984149e-03 -5.67803439e-03]
  ...
  [-1.22438511e-03  5.26216580e-04 -1.09949242e-02 ... -8.12670449e-04
    5.85858990e-03 -6.60686754e-03]
  [-5.36383037e-03  2.80846469e-03 -9.67217842e-04 ... -7.30457250e-04
    4.82074078e-03 -6.63320953e-03]
  [-5.37302252e-03  2.37890705e-03  5.01788873e-03 ... -2.24496517e-03
    9.32508521e-03 -2.93301791e-03]]

 [[-1.52882468e-03 -6.09554874e-04 -1.14973858e-02 ...  2.86858471e-04
    1.18946086e-03  2.47103651e-03]
  [ 4.91021259e-04 -1.29920198e-04 -8.80455039e-03 ... -1.97776081e-03
    5.27392700e-03  3.99522251e-04]
  [-4.04006569e-04 -1.97579851e-04 -1.04782823e-02 ... -2.79055024e-03
    1.37431244e-03 -1.98141579e-03]
  ...
  [-3.96271050e-03  7.05333985e-03 -6.04499876e-03 ...  2.62893178e-03
    1.12824719e-02  4.66081453e-03]
```

```
  [-9.76548344e-03  1.05691031e-02 -6.62725884e-04 ...  7.44159985e-03
    6.00423664e-03  1.49577064e-03]
  [-1.45169925e-02  7.26563437e-03 -8.20361194e-04 ...  1.15945563e-02
    3.13329999e-03 -7.39687262e-03]]

 [[-2.93885334e-03  2.50824308e-03  7.25660659e-03 ...  2.71491765e-04
   -4.13605245e-04  4.47312952e-04]
  [ 1.53860997e-03 -1.50424172e-03  2.43748119e-03 ... -1.86618557e-03
   -1.85876875e-03  3.63257434e-03]
  [-4.84319497e-03  4.07647248e-03  5.07520186e-03 ...  2.53993273e-03
   -3.52198025e-03  1.68331130e-03]
  ...
  [-1.17551573e-02  2.30930280e-04 -5.90215763e-03 ...  3.25255701e-03
    1.28287477e-02 -1.50992665e-02]
  [-1.43166659e-02 -5.11382148e-03 -4.12890455e-03 ...  3.07356729e-03
    1.43895820e-02 -7.92813487e-03]
  [-1.46746216e-02 -8.89127702e-03 -3.64007754e-03 ...  2.01895204e-03
    1.53979510e-02 -2.11373740e-03]]], shape=(64, 100, 65),
dtype=float32)
```

```
pred = example_batch_predictions[0]
print(len(pred))
print(pred)
```

```
100
tf.Tensor(
[[-6.0533732e-03  6.0662590e-03 -1.6988590e-05 ... -2.2828502e-03
  -4.2157057e-03 -2.6675304e-03]
 [-5.2381642e-03  4.6179369e-03 -1.1760829e-02 ... -1.4432358e-03
  -1.8208690e-03  9.4690523e-04]
 [-9.8757623e-03  9.6103940e-03 -9.2857629e-03 ... -3.2211756e-03
  -4.7619576e-03 -1.4286945e-03]
 ...
 [-7.2728759e-03  3.8498687e-03 -1.8318413e-02 ... -1.5916311e-03
   3.2518967e-03 -7.5457841e-03]
 [-5.9286272e-03  1.1068466e-03 -1.8462390e-02 ...  7.6053722e-04
  -2.3716851e-04 -7.0589767e-03]
 [-4.4758013e-03 -3.2166747e-04 -8.2820728e-03 ... -8.9906063e-04
   5.4971557e-03 -2.6553620e-03]], shape=(100, 65), dtype=float32)
```

```
time_pred = pred[0]
print(len(time_pred))
print(time_pred)
```

```
65
tf.Tensor(
[-6.0533732e-03  6.0662590e-03 -1.6988590e-05 -4.6567025e-04
 -5.0640185e-03 -8.2127303e-03 -4.8505026e-04 -1.3435832e-03
  5.3496808e-03  1.8426554e-03 -1.0210044e-03 -2.3347288e-03
  1.0121699e-03  7.0214849e-03  1.5197438e-05  1.1496756e-03
  5.3773588e-03 -3.4622804e-03 -1.4127383e-03 -2.5830460e-03
```

```
 -4.6662916e-04 -4.2427666e-03  7.2445106e-03 -4.0093604e-03
  2.7526047e-03 -1.0857212e-03 -4.6243672e-03 -6.8200962e-04
  1.2732418e-03 -3.6708547e-03  8.6831010e-04 -7.5767632e-05
 -1.7732610e-03  4.1059754e-04  1.6342072e-03 -1.7973820e-03
  2.4138936e-03 -2.8343864e-03  1.7433002e-03  1.8572274e-03
  4.9016112e-03 -3.3892305e-03  1.1348494e-03 -7.8401470e-04
 -6.6295392e-03 -5.6314887e-04 -3.1256292e-03 -4.5251031e-04
 -3.8937535e-03 -7.9109240e-04  9.9687628e-04 -3.2813959e-03
  8.0943219e-03  2.0723217e-03  3.1089475e-03 -2.3531434e-03
 -2.6628356e-03  4.2445278e-03 -3.0891844e-03  2.5795060e-03
  3.4656376e-04  1.5732367e-04 -2.2828502e-03 -4.2157057e-03
 -2.6675304e-03], shape=(65,), dtype=float32)
```

```
sampled_indices = tf.random.categorical(pred, num_samples = 1)
sampled_indices = np.reshape(sampled_indices, (1,-1))[0]
predicted_chars = int_to_text(sampled_indices)

predicted_chars
```

```
"PmHlfoCto-a?'OyOH-GHbYKlJp'AZBobk.h.&&o!Bre3oIjONyefq?
x'uGkN&.grN$wA:nf$ TXNBDi,rE.ntkR$YRPij$N;vszw"
```

```
def loss(labels,logits):
  return
tf.keras.losses.sparse_categorical_crossentropy(labels,logits,from_log
its= True)

model.compile(optimizer = 'adam', loss = loss, metrics = ['accuracy'])
```

Early stop , Checkpoint and LRonPlateau

```
filepath = '/content/sample_data/lstm.h5'
checkpoint = ModelCheckpoint(filepath,monitor = 'loss', verbose = 1,
save_best_only = True, mode = 'min')
early_stop=EarlyStopping(monitor='loss',mode='min',verbose=1,patience=
5)
lr_reduce = ReduceLROnPlateau(monitor='accuracy',
factor=0.6,patience=5, verbose=1, mode='max', min_lr=0.001)
callbacklist = [lr_reduce,checkpoint,early_stop]
```

Fitting the model

```
history = model.fit(data, epochs = 100, callbacks = callbacklist)
```

```
Epoch 1/100
172/172 [==============================] - 12s 60ms/step - loss:
2.5869 - accuracy: 0.2910

Epoch 00001: loss improved from inf to 2.58688, saving model to
/content/sample_data/lstm.h5
Epoch 2/100
172/172 [==============================] - 11s 61ms/step - loss:
1.9104 - accuracy: 0.4424
```

```
Epoch 00002: loss improved from 2.58688 to 1.91041, saving model to
/content/sample_data/lstm.h5
Epoch 3/100
172/172 [==============================] - 10s 60ms/step - loss:
1.6718 - accuracy: 0.5063

Epoch 00003: loss improved from 1.91041 to 1.67183, saving model to
/content/sample_data/lstm.h5
Epoch 4/100
172/172 [==============================] - 10s 60ms/step - loss:
1.5318 - accuracy: 0.5428

Epoch 00004: loss improved from 1.67183 to 1.53177, saving model to
/content/sample_data/lstm.h5
Epoch 5/100
172/172 [==============================] - 10s 60ms/step - loss:
1.4459 - accuracy: 0.5649

Epoch 00005: loss improved from 1.53177 to 1.44594, saving model to
/content/sample_data/lstm.h5
Epoch 6/100
172/172 [==============================] - 11s 61ms/step - loss:
1.3870 - accuracy: 0.5801

Epoch 00006: loss improved from 1.44594 to 1.38700, saving model to
/content/sample_data/lstm.h5
Epoch 7/100
172/172 [==============================] - 10s 59ms/step - loss:
1.3412 - accuracy: 0.5921

Epoch 00007: loss improved from 1.38700 to 1.34118, saving model to
/content/sample_data/lstm.h5
Epoch 8/100
172/172 [==============================] - 11s 61ms/step - loss:
1.3032 - accuracy: 0.6021

Epoch 00008: loss improved from 1.34118 to 1.30317, saving model to
/content/sample_data/lstm.h5
Epoch 9/100
172/172 [==============================] - 10s 60ms/step - loss:
1.2687 - accuracy: 0.6115

Epoch 00009: loss improved from 1.30317 to 1.26874, saving model to
/content/sample_data/lstm.h5
Epoch 10/100
172/172 [==============================] - 10s 60ms/step - loss:
1.2358 - accuracy: 0.6201
```

```
Epoch 00010: loss improved from 1.26874 to 1.23577, saving model to
/content/sample_data/lstm.h5
Epoch 11/100
172/172 [==============================] - 11s 61ms/step - loss:
1.2036 - accuracy: 0.6296

Epoch 00011: loss improved from 1.23577 to 1.20362, saving model to
/content/sample_data/lstm.h5
Epoch 12/100
172/172 [==============================] - 10s 60ms/step - loss:
1.1706 - accuracy: 0.6390

Epoch 00012: loss improved from 1.20362 to 1.17059, saving model to
/content/sample_data/lstm.h5
Epoch 13/100
172/172 [==============================] - 10s 60ms/step - loss:
1.1369 - accuracy: 0.6497

Epoch 00013: loss improved from 1.17059 to 1.13693, saving model to
/content/sample_data/lstm.h5
Epoch 14/100
172/172 [==============================] - 11s 61ms/step - loss:
1.1030 - accuracy: 0.6603

Epoch 00014: loss improved from 1.13693 to 1.10304, saving model to
/content/sample_data/lstm.h5
Epoch 15/100
172/172 [==============================] - 10s 60ms/step - loss:
1.0670 - accuracy: 0.6718

Epoch 00015: loss improved from 1.10304 to 1.06700, saving model to
/content/sample_data/lstm.h5
Epoch 16/100
172/172 [==============================] - 10s 60ms/step - loss:
1.0305 - accuracy: 0.6840

Epoch 00016: loss improved from 1.06700 to 1.03046, saving model to
/content/sample_data/lstm.h5
Epoch 17/100
172/172 [==============================] - 11s 60ms/step - loss:
0.9938 - accuracy: 0.6964

Epoch 00017: loss improved from 1.03046 to 0.99382, saving model to
/content/sample_data/lstm.h5
Epoch 18/100
172/172 [==============================] - 10s 59ms/step - loss:
0.9572 - accuracy: 0.7093

Epoch 00018: loss improved from 0.99382 to 0.95718, saving model to
```

```
/content/sample_data/lstm.h5
Epoch 19/100
172/172 [==============================] - 10s 59ms/step - loss:
0.9202 - accuracy: 0.7226

Epoch 00019: loss improved from 0.95718 to 0.92021, saving model to
/content/sample_data/lstm.h5
Epoch 20/100
172/172 [==============================] - 10s 60ms/step - loss:
0.8858 - accuracy: 0.7348

Epoch 00020: loss improved from 0.92021 to 0.88583, saving model to
/content/sample_data/lstm.h5
Epoch 21/100
172/172 [==============================] - 10s 60ms/step - loss:
0.8518 - accuracy: 0.7474

Epoch 00021: loss improved from 0.88583 to 0.85178, saving model to
/content/sample_data/lstm.h5
Epoch 22/100
172/172 [==============================] - 10s 60ms/step - loss:
0.8193 - accuracy: 0.7594

Epoch 00022: loss improved from 0.85178 to 0.81928, saving model to
/content/sample_data/lstm.h5
Epoch 23/100
172/172 [==============================] - 10s 60ms/step - loss:
0.7884 - accuracy: 0.7710

Epoch 00023: loss improved from 0.81928 to 0.78837, saving model to
/content/sample_data/lstm.h5
Epoch 24/100
172/172 [==============================] - 11s 61ms/step - loss:
0.7608 - accuracy: 0.7808

Epoch 00024: loss improved from 0.78837 to 0.76077, saving model to
/content/sample_data/lstm.h5
Epoch 25/100
172/172 [==============================] - 10s 60ms/step - loss:
0.7339 - accuracy: 0.7910

Epoch 00025: loss improved from 0.76077 to 0.73386, saving model to
/content/sample_data/lstm.h5
Epoch 26/100
172/172 [==============================] - 10s 60ms/step - loss:
0.7111 - accuracy: 0.7995

Epoch 00026: loss improved from 0.73386 to 0.71106, saving model to
/content/sample_data/lstm.h5
```

```
Epoch 27/100
172/172 [==============================] - 11s 61ms/step - loss:
0.6885 - accuracy: 0.8080

Epoch 00027: loss improved from 0.71106 to 0.68847, saving model to
/content/sample_data/lstm.h5
Epoch 28/100
172/172 [==============================] - 10s 59ms/step - loss:
0.6671 - accuracy: 0.8161

Epoch 00028: loss improved from 0.68847 to 0.66714, saving model to
/content/sample_data/lstm.h5
Epoch 29/100
172/172 [==============================] - 10s 59ms/step - loss:
0.6494 - accuracy: 0.8228

Epoch 00029: loss improved from 0.66714 to 0.64940, saving model to
/content/sample_data/lstm.h5
Epoch 30/100
172/172 [==============================] - 10s 60ms/step - loss:
0.6323 - accuracy: 0.8291

Epoch 00030: loss improved from 0.64940 to 0.63230, saving model to
/content/sample_data/lstm.h5
Epoch 31/100
172/172 [==============================] - 11s 60ms/step - loss:
0.6153 - accuracy: 0.8354

Epoch 00031: loss improved from 0.63230 to 0.61527, saving model to
/content/sample_data/lstm.h5
Epoch 32/100
172/172 [==============================] - 10s 60ms/step - loss:
0.6023 - accuracy: 0.8403

Epoch 00032: loss improved from 0.61527 to 0.60234, saving model to
/content/sample_data/lstm.h5
Epoch 33/100
172/172 [==============================] - 10s 60ms/step - loss:
0.5889 - accuracy: 0.8451

Epoch 00033: loss improved from 0.60234 to 0.58890, saving model to
/content/sample_data/lstm.h5
Epoch 34/100
172/172 [==============================] - 10s 60ms/step - loss:
0.5762 - accuracy: 0.8496

Epoch 00034: loss improved from 0.58890 to 0.57621, saving model to
/content/sample_data/lstm.h5
Epoch 35/100
```

```
172/172 [==============================] - 11s 60ms/step - loss:
0.5650 - accuracy: 0.8538

Epoch 00035: loss improved from 0.57621 to 0.56497, saving model to
/content/sample_data/lstm.h5
Epoch 36/100
172/172 [==============================] - 10s 60ms/step - loss:
0.5548 - accuracy: 0.8580

Epoch 00036: loss improved from 0.56497 to 0.55478, saving model to
/content/sample_data/lstm.h5
Epoch 37/100
172/172 [==============================] - 11s 60ms/step - loss:
0.5446 - accuracy: 0.8614

Epoch 00037: loss improved from 0.55478 to 0.54456, saving model to
/content/sample_data/lstm.h5
Epoch 38/100
172/172 [==============================] - 10s 59ms/step - loss:
0.5362 - accuracy: 0.8644

Epoch 00038: loss improved from 0.54456 to 0.53617, saving model to
/content/sample_data/lstm.h5
Epoch 39/100
172/172 [==============================] - 10s 60ms/step - loss:
0.5281 - accuracy: 0.8671

Epoch 00039: loss improved from 0.53617 to 0.52811, saving model to
/content/sample_data/lstm.h5
Epoch 40/100
172/172 [==============================] - 11s 60ms/step - loss:
0.5194 - accuracy: 0.8700

Epoch 00040: loss improved from 0.52811 to 0.51945, saving model to
/content/sample_data/lstm.h5
Epoch 41/100
172/172 [==============================] - 10s 59ms/step - loss:
0.5137 - accuracy: 0.8724

Epoch 00041: loss improved from 0.51945 to 0.51373, saving model to
/content/sample_data/lstm.h5
Epoch 42/100
172/172 [==============================] - 11s 59ms/step - loss:
0.5080 - accuracy: 0.8747

Epoch 00042: loss improved from 0.51373 to 0.50800, saving model to
/content/sample_data/lstm.h5
Epoch 43/100
172/172 [==============================] - 11s 60ms/step - loss:
```

```
0.5021 - accuracy: 0.8767

Epoch 00043: loss improved from 0.50800 to 0.50207, saving model to
/content/sample_data/lstm.h5
Epoch 44/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4972 - accuracy: 0.8782

Epoch 00044: loss improved from 0.50207 to 0.49725, saving model to
/content/sample_data/lstm.h5
Epoch 45/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4920 - accuracy: 0.8800

Epoch 00045: loss improved from 0.49725 to 0.49199, saving model to
/content/sample_data/lstm.h5
Epoch 46/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4866 - accuracy: 0.8820

Epoch 00046: loss improved from 0.49199 to 0.48661, saving model to
/content/sample_data/lstm.h5
Epoch 47/100
172/172 [==============================] - 11s 61ms/step - loss:
0.4833 - accuracy: 0.8829

Epoch 00047: loss improved from 0.48661 to 0.48327, saving model to
/content/sample_data/lstm.h5
Epoch 48/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4766 - accuracy: 0.8863

Epoch 00048: loss improved from 0.48327 to 0.47656, saving model to
/content/sample_data/lstm.h5
Epoch 49/100
172/172 [==============================] - 10s 59ms/step - loss:
0.4719 - accuracy: 0.8874

Epoch 00049: loss improved from 0.47656 to 0.47191, saving model to
/content/sample_data/lstm.h5
Epoch 50/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4686 - accuracy: 0.8882

Epoch 00050: loss improved from 0.47191 to 0.46861, saving model to
/content/sample_data/lstm.h5
Epoch 51/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4673 - accuracy: 0.8886
```

```
Epoch 00051: loss improved from 0.46861 to 0.46729, saving model to
/content/sample_data/lstm.h5
Epoch 52/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4626 - accuracy: 0.8900

Epoch 00052: loss improved from 0.46729 to 0.46255, saving model to
/content/sample_data/lstm.h5
Epoch 53/100
172/172 [==============================] - 10s 59ms/step - loss:
0.4614 - accuracy: 0.8904

Epoch 00053: loss improved from 0.46255 to 0.46136, saving model to
/content/sample_data/lstm.h5
Epoch 54/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4564 - accuracy: 0.8924

Epoch 00054: loss improved from 0.46136 to 0.45642, saving model to
/content/sample_data/lstm.h5
Epoch 55/100
172/172 [==============================] - 11s 61ms/step - loss:
0.4522 - accuracy: 0.8933

Epoch 00055: loss improved from 0.45642 to 0.45219, saving model to
/content/sample_data/lstm.h5
Epoch 56/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4518 - accuracy: 0.8941

Epoch 00056: loss improved from 0.45219 to 0.45182, saving model to
/content/sample_data/lstm.h5
Epoch 57/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4480 - accuracy: 0.8947

Epoch 00057: loss improved from 0.45182 to 0.44800, saving model to
/content/sample_data/lstm.h5
Epoch 58/100
172/172 [==============================] - 11s 61ms/step - loss:
0.4444 - accuracy: 0.8960

Epoch 00058: loss improved from 0.44800 to 0.44441, saving model to
/content/sample_data/lstm.h5
Epoch 59/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4429 - accuracy: 0.8967
```

```
Epoch 00059: loss improved from 0.44441 to 0.44286, saving model to
/content/sample_data/lstm.h5
Epoch 60/100
172/172 [==============================] - 11s 61ms/step - loss:
0.4413 - accuracy: 0.8974

Epoch 00060: loss improved from 0.44286 to 0.44130, saving model to
/content/sample_data/lstm.h5
Epoch 61/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4372 - accuracy: 0.8984

Epoch 00061: loss improved from 0.44130 to 0.43719, saving model to
/content/sample_data/lstm.h5
Epoch 62/100
172/172 [==============================] - 10s 59ms/step - loss:
0.4377 - accuracy: 0.8986

Epoch 00062: loss did not improve from 0.43719
Epoch 63/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4375 - accuracy: 0.8990

Epoch 00063: loss did not improve from 0.43719
Epoch 64/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4337 - accuracy: 0.8993

Epoch 00064: loss improved from 0.43719 to 0.43368, saving model to
/content/sample_data/lstm.h5
Epoch 65/100
172/172 [==============================] - 11s 61ms/step - loss:
0.4308 - accuracy: 0.9005

Epoch 00065: loss improved from 0.43368 to 0.43080, saving model to
/content/sample_data/lstm.h5
Epoch 66/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4320 - accuracy: 0.9001

Epoch 00066: loss did not improve from 0.43080
Epoch 67/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4307 - accuracy: 0.9003

Epoch 00067: loss improved from 0.43080 to 0.43071, saving model to
/content/sample_data/lstm.h5
Epoch 68/100
172/172 [==============================] - 10s 60ms/step - loss:
```

```
0.4302 - accuracy: 0.9008

Epoch 00068: loss improved from 0.43071 to 0.43018, saving model to
/content/sample_data/lstm.h5
Epoch 69/100
172/172 [==============================] - 11s 61ms/step - loss:
0.4268 - accuracy: 0.9014

Epoch 00069: loss improved from 0.43018 to 0.42682, saving model to
/content/sample_data/lstm.h5
Epoch 70/100
172/172 [==============================] - 10s 59ms/step - loss:
0.4228 - accuracy: 0.9027

Epoch 00070: loss improved from 0.42682 to 0.42283, saving model to
/content/sample_data/lstm.h5
Epoch 71/100
172/172 [==============================] - 11s 61ms/step - loss:
0.4237 - accuracy: 0.9023

Epoch 00071: loss did not improve from 0.42283
Epoch 72/100
172/172 [==============================] - 10s 59ms/step - loss:
0.4221 - accuracy: 0.9031

Epoch 00072: loss improved from 0.42283 to 0.42215, saving model to
/content/sample_data/lstm.h5
Epoch 73/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4222 - accuracy: 0.9030

Epoch 00073: loss did not improve from 0.42215
Epoch 74/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4204 - accuracy: 0.9029

Epoch 00074: loss improved from 0.42215 to 0.42043, saving model to
/content/sample_data/lstm.h5
Epoch 75/100
172/172 [==============================] - 10s 59ms/step - loss:
0.4191 - accuracy: 0.9035

Epoch 00075: loss improved from 0.42043 to 0.41905, saving model to
/content/sample_data/lstm.h5
Epoch 76/100
172/172 [==============================] - 11s 60ms/step - loss:
0.4186 - accuracy: 0.9037

Epoch 00076: loss improved from 0.41905 to 0.41856, saving model to
```

```
/content/sample_data/lstm.h5
Epoch 77/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4170 - accuracy: 0.9040

Epoch 00077: loss improved from 0.41856 to 0.41697, saving model to
/content/sample_data/lstm.h5
Epoch 78/100
172/172 [==============================] - 11s 61ms/step - loss:
0.4203 - accuracy: 0.9029

Epoch 00078: loss did not improve from 0.41697
Epoch 79/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4205 - accuracy: 0.9026

Epoch 00079: loss did not improve from 0.41697
Epoch 80/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4201 - accuracy: 0.9030

Epoch 00080: loss did not improve from 0.41697
Epoch 81/100
172/172 [==============================] - 10s 59ms/step - loss:
0.4155 - accuracy: 0.9043

Epoch 00081: loss improved from 0.41697 to 0.41555, saving model to
/content/sample_data/lstm.h5
Epoch 82/100
172/172 [==============================] - 10s 59ms/step - loss:
0.4126 - accuracy: 0.9057

Epoch 00082: loss improved from 0.41555 to 0.41261, saving model to
/content/sample_data/lstm.h5
Epoch 83/100
172/172 [==============================] - 11s 60ms/step - loss:
0.4120 - accuracy: 0.9054

Epoch 00083: loss improved from 0.41261 to 0.41205, saving model to
/content/sample_data/lstm.h5
Epoch 84/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4121 - accuracy: 0.9052

Epoch 00084: loss did not improve from 0.41205
Epoch 85/100
172/172 [==============================] - 11s 60ms/step - loss:
0.4115 - accuracy: 0.9053
```

```
Epoch 00085: loss improved from 0.41205 to 0.41151, saving model to
/content/sample_data/lstm.h5
Epoch 86/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4098 - accuracy: 0.9063

Epoch 00086: loss improved from 0.41151 to 0.40979, saving model to
/content/sample_data/lstm.h5
Epoch 87/100
172/172 [==============================] - 11s 60ms/step - loss:
0.4086 - accuracy: 0.9065

Epoch 00087: loss improved from 0.40979 to 0.40859, saving model to
/content/sample_data/lstm.h5
Epoch 88/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4082 - accuracy: 0.9066

Epoch 00088: loss improved from 0.40859 to 0.40821, saving model to
/content/sample_data/lstm.h5
Epoch 89/100
172/172 [==============================] - 10s 60ms/step - loss:
0.4070 - accuracy: 0.9064

Epoch 00089: loss improved from 0.40821 to 0.40696, saving model to
/content/sample_data/lstm.h5
Epoch 90/100
172/172 [==============================] - 11s 61ms/step - loss:
0.4069 - accuracy: 0.9061

Epoch 00090: loss improved from 0.40696 to 0.40688, saving model to
/content/sample_data/lstm.h5
Epoch 91/100
172/172 [==============================] - 10s 59ms/step - loss:
0.4096 - accuracy: 0.9054

Epoch 00091: loss did not improve from 0.40688
Epoch 92/100
172/172 [==============================] - 11s 60ms/step - loss:
0.4096 - accuracy: 0.9051

Epoch 00092: loss did not improve from 0.40688
Epoch 93/100
172/172 [==============================] - 11s 60ms/step - loss:
0.4086 - accuracy: 0.9058

Epoch 00093: loss did not improve from 0.40688
Epoch 94/100
172/172 [==============================] - 10s 60ms/step - loss:
```

```
0.4101 - accuracy: 0.9053

Epoch 00094: loss did not improve from 0.40688
Epoch 95/100
172/172 [==============================] - 11s 61ms/step - loss:
0.4082 - accuracy: 0.9058

Epoch 00095: loss did not improve from 0.40688
Epoch 00095: early stopping
```

The model ran for 95 epochs in my case as the loss did not decrease it stops

Building the model for generating text

```
model = build_model(VOCAB_SIZE, EMBEDDING_DIM, RNN_UNITS, batch_size =
1)

model.load_weights(filepath)
model.build(tf.TensorShape([1,None]))
```

Generate text

```
def generate_text(model, start_string):
  num_generate = 800
  input_eval = [char2idx[s] for s in start_string]
  input_eval = tf.expand_dims(input_eval,0)

  text_generated = []

  temp= 1.0

  model.reset_states()
  for i in range(num_generate):
    predictions = model(input_eval)
    predictions = tf.squeeze(predictions,0)
    predictions = predictions/temp
    predicted_id = tf.random.categorical(predictions, num_samples = 1)
[-1,0].numpy()
    input_eval = tf.expand_dims([predicted_id], 0)
    text_generated.append(idx2char[predicted_id])

  return(start_string + ''.join(text_generated))

inp = input('Type a starting string: ')
print(generate_text(model,inp))
```

```
Type a starting string:  romeo

romeough the princes have betured to do so.

Provost:
In good times strings before a usurer; it is done,
And then, to purge full of itself, but with the
```

```
intedious faults, and is not Half-can tyranny:
To make a pelibate and goodness on thee,
Being crown'd the age us to incur you come; for then I play thee go, a
matter and a coward, a tell thou art.

GLOUCESTER:
But shall I live on nature, of a sister God presently
Hath your employment in my way,
Knowing a tardy world, an humbly thanks.

GLOUCESTER:
Ah! at once we stop all her beseeching greater them.

BIANCA:
The two right suprements my son should be
The freshest things now of the midnight
Shall have a high and supper in Baptista.

BAPTISTA:
Good my lord, good night: I know it please you did thream.

HORTENSIO:
I am content.

MARCIUS:
Well, sir,
```

Plotting the graphs

```
import matplotlib.pyplot as plt

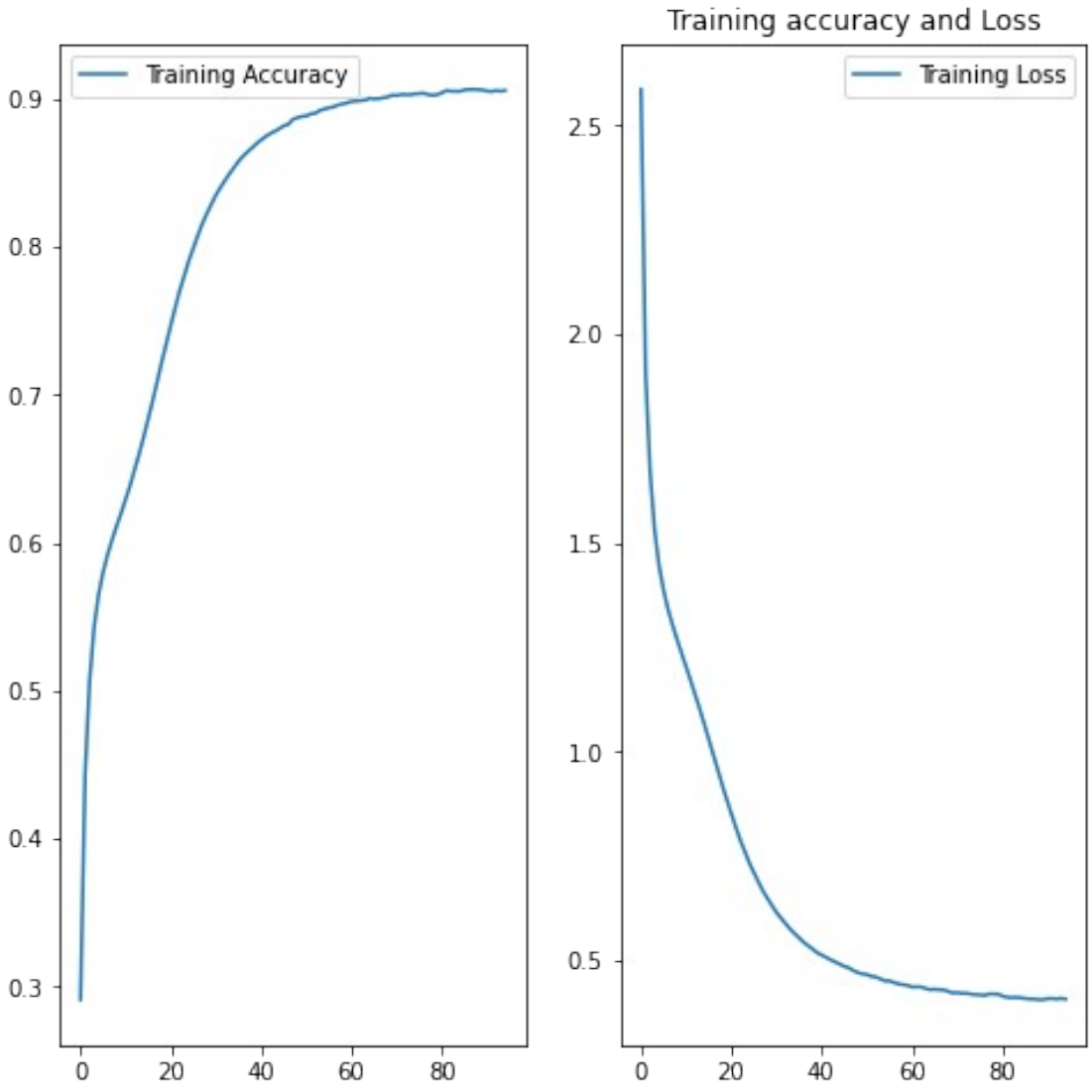
acc = history.history['accuracy']

loss = history.history['loss']

epochs_range = range(95)
plt.figure(figsize=(8, 8))
plt.subplot(1,2,1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.legend(loc='best')
plt.subplot(1,2,2)
plt.plot(epochs_range, loss, label='Training Loss')

plt.title('Training and Validation Accuracy')
plt.legend(loc='best')

plt.title('Training accuracy and Loss')
plt.show()
```


Training accuracy and Loss
Training Accuracy
Training Loss
0.9
0.8
0.7
0.6
0.5
0.4
0.3
2.5
2.0
1.5
1.0
0.5
0
20
40
60
80